SDD

ONLINE VOTING SYSTEM

MUHAMMAD JUNAID
DIT

Software Design Document

Blockchain-Based Secure Online Voting System

Submitted by:

MUHAMMD JUNAID**: S23BINFT1M01013**

Submitted to:

Mam. MUSSARAT KARIM

Department of Information Technology

Faculty of Computing

The Islamia University of Bahawalpur.

1. Introduction

1.1 Purpose

This Software Design Document (SDD) provides a complete architectural, component-level, data, interface, and interaction design for the Blockchain-Based Secure Online Voting System. It serves as a detailed blueprint for development based on the SRS.

1.2 Scope

The document covers the design of smart contracts, frontend components, blockchain interaction, data structures, user interfaces, and key workflows.

1.3 References

- Software Requirements Specification (SRS)
- FYP Proposal
- Solidity, Ethers.js, and React.js Documentation

2. Design Goals and Principles

- Security & Immutability: Tamper-proof voting through blockchain and smart contracts.
- Transparency: All votes publicly verifiable.
- Anonymity: Wallet-based authentication without revealing real identity.
- Usability: Intuitive and responsive design.
- Modularity & Maintainability: Clean separation of concerns.
- Gas Efficiency: Optimized smart contracts for testnet.

3. System Architecture

3.1 High-Level Architecture

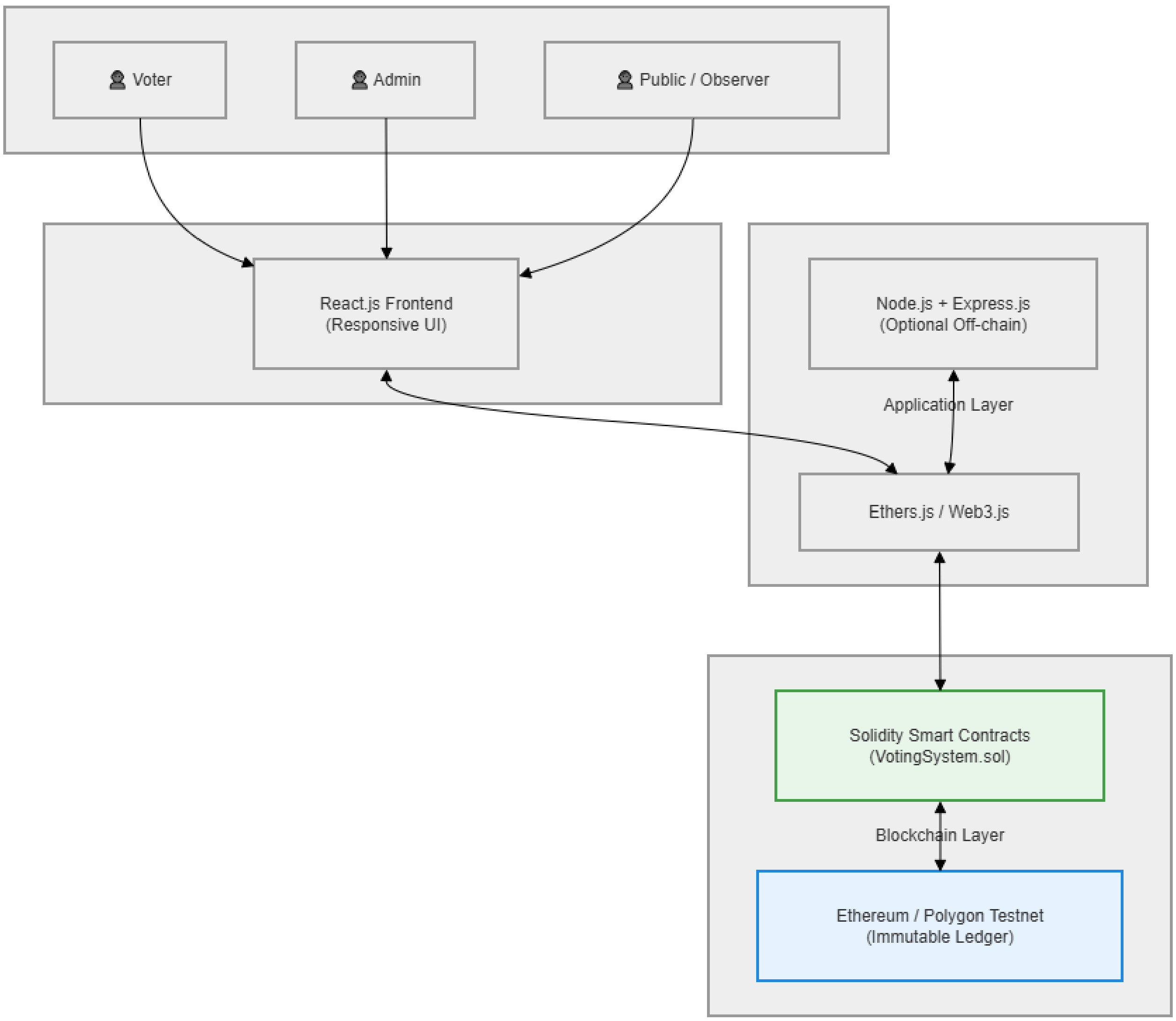
The system follows a Three-Tier Decentralized Architecture:

- Presentation Layer – React.js Frontend
- Application Layer – Ethers.js/Web3.js + Optional Node.js
- Blockchain Layer – Solidity Smart Contracts on Polygon/Ethereum testnet

Figure 3.1: High-Level System Architecture Diagram

Description:

This diagram illustrates the three-tier architecture of the Blockchain-Based Secure Online Voting System. Users interact with the React.js frontend, which communicates with the blockchain through Ethers.js. All voting logic and data are handled by Solidity smart contracts deployed on the testnet.



4. Component Design

4.1 Smart Contract Design

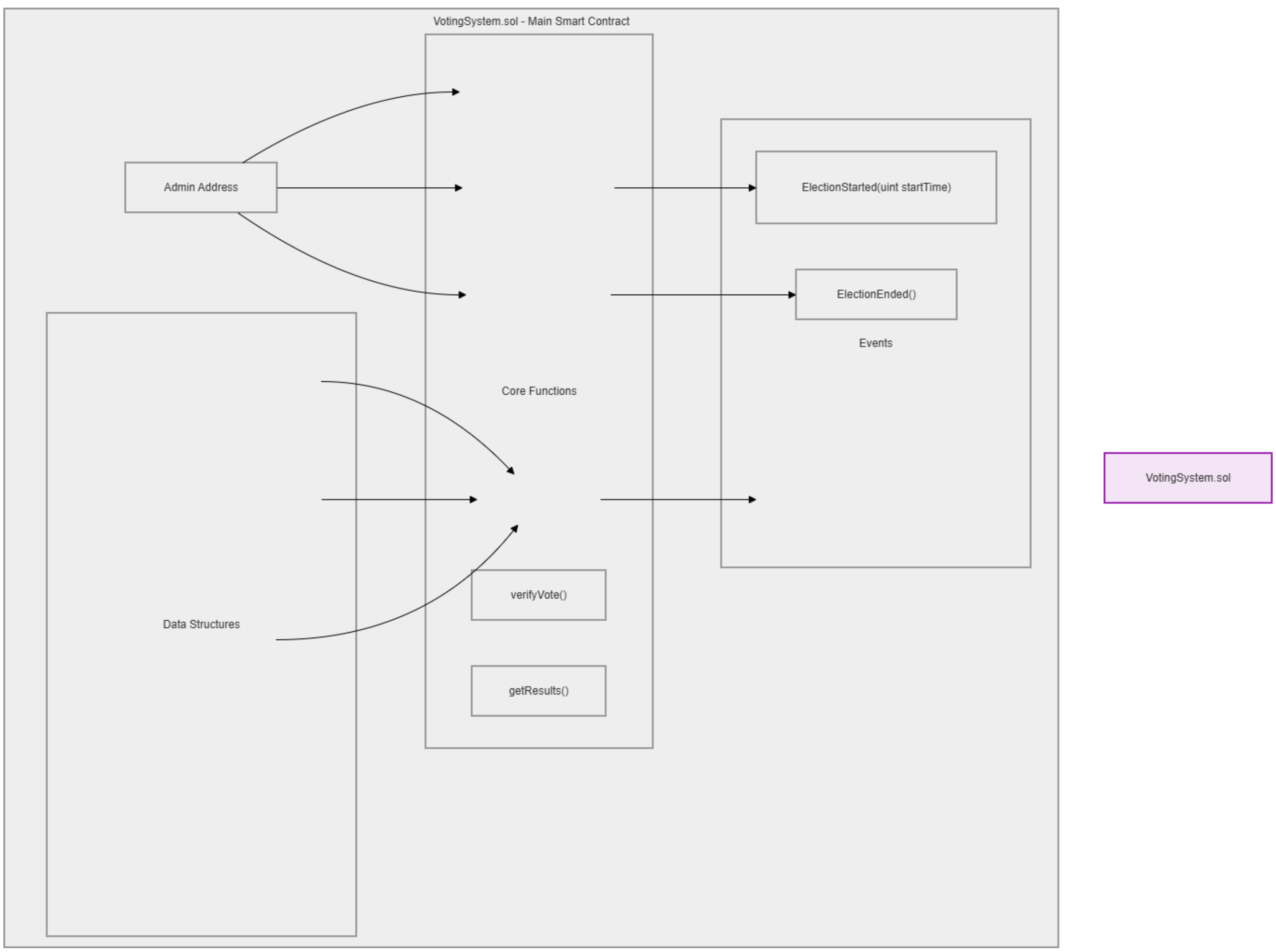
Contract Name: VotingSystem.sol

Key Data Structures & Functions are defined as per SRS.

Figure 4.1: Smart Contract Structure Diagram

Description:

This diagram shows the internal structure of the main VotingSystem.sol smart contract, including key data structures (Candidate struct, mappings), core functions, and emitted events. It represents the on-chain logic responsible for secure vote casting and election management.



4.2 Frontend Component Design

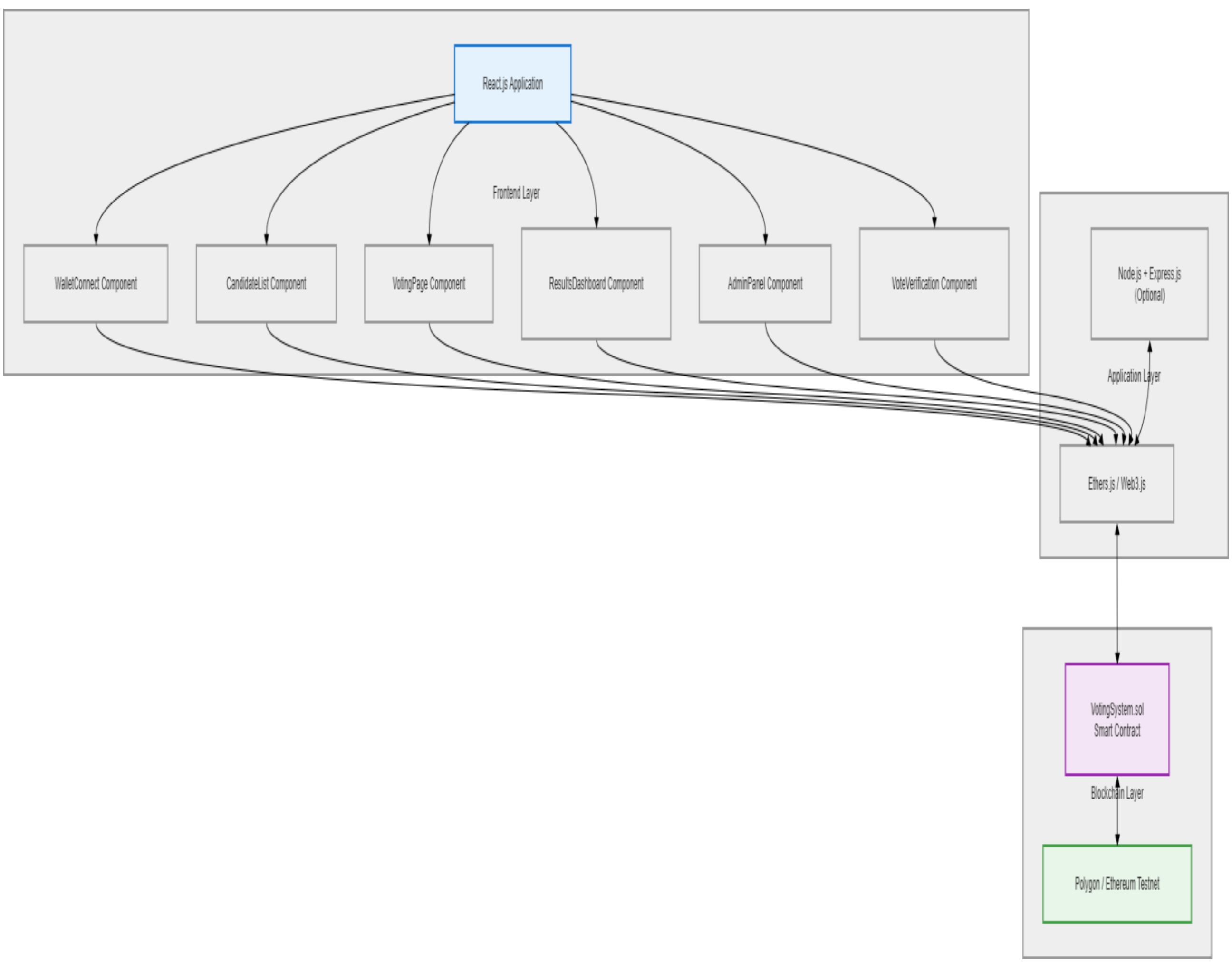
Main React Components:

- WalletConnect
- CandidateCard
- VotingPage
- ResultsDashboard
- AdminPanel
- VoteVerification

Figure 4.2: Component Diagram

Description:

This diagram shows the main components of the Blockchain-Based Secure Online Voting System. The React.js frontend components interact with the blockchain through Ethers.js, while the core logic resides in the Solidity smart contract.

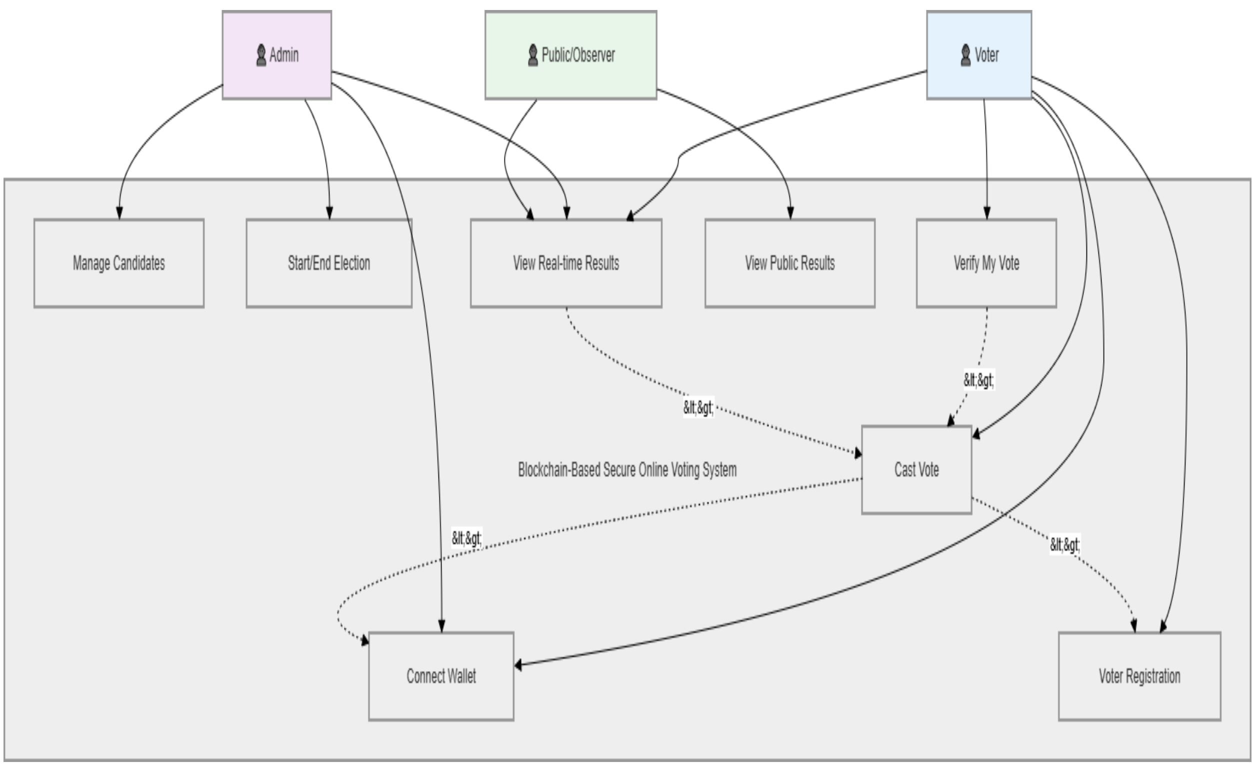


4.3 Use Case Realization

Figure 4.3: Use Case Diagram

Description:

This diagram illustrates the interactions between the main actors (Voter, Admin, and Public/Observer) and the key use cases of the system. The "Cast Vote" use case includes wallet connection and voter registration, while "Verify My Vote" and "View Real-time Results" extend from it.



5. Data Design

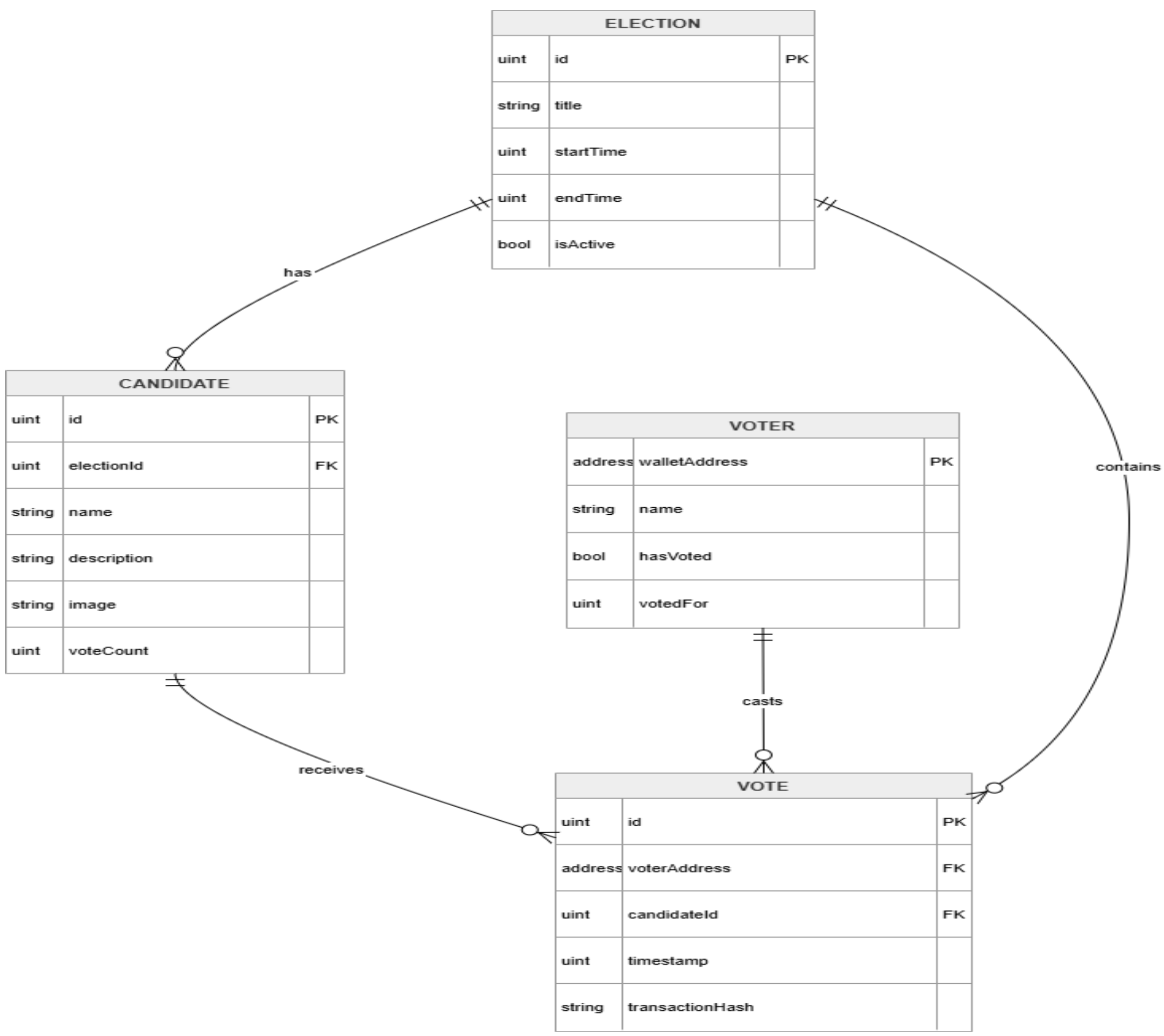
5.1 On-Chain Data Model

Critical data (voters, candidates, election state, votes) is stored on the blockchain.

Figure 5.1: Entity Relationship Diagram (ERD)

Description:

This diagram represents the on-chain data model of the Blockchain-Based Secure Online Voting System. It shows the main entities (Election, Candidate, Voter, and Vote) and their relationships. All critical data is stored immutably on the blockchain through smart contracts.



6. User Interface Design

Design Principles:

Responsive, modern blue/green color scheme, clean and trustworthy look.

Main Interfaces:

- Landing / Wallet Connection Page
- Voter Dashboard & Candidate Selection
- Voting Confirmation Page
- Live Results Dashboard
- Admin Management Panel

Figure 6.1: UI Wireframe – Home / Connect Wallet Page

Description:

This wireframe shows the landing page of the system. It features a clean hero section with a prominent "Connect Wallet" button using MetaMask. The design follows a modern, trustworthy blue/green color scheme

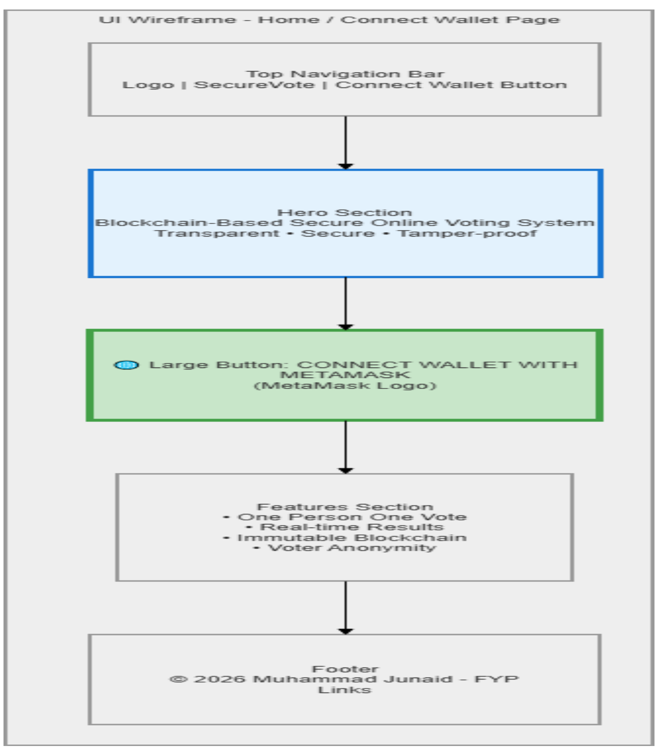


Figure 6.2: UI Wireframe – Voting Page

Description:

This wireframe shows the main voting interface where the voter can see all candidates with their details and vote buttons. The design emphasizes clarity, one-person-one-vote rule, and easy navigation

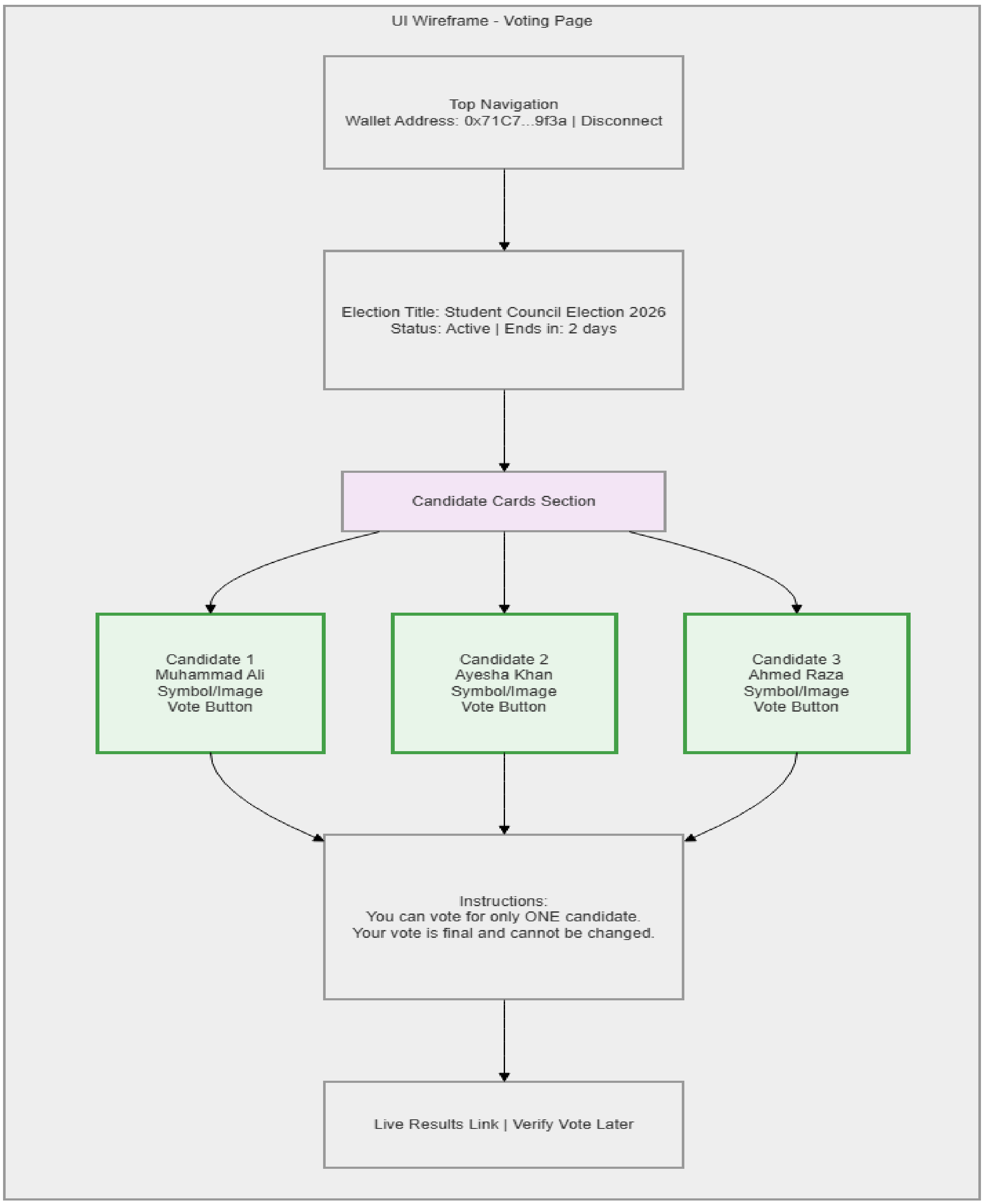
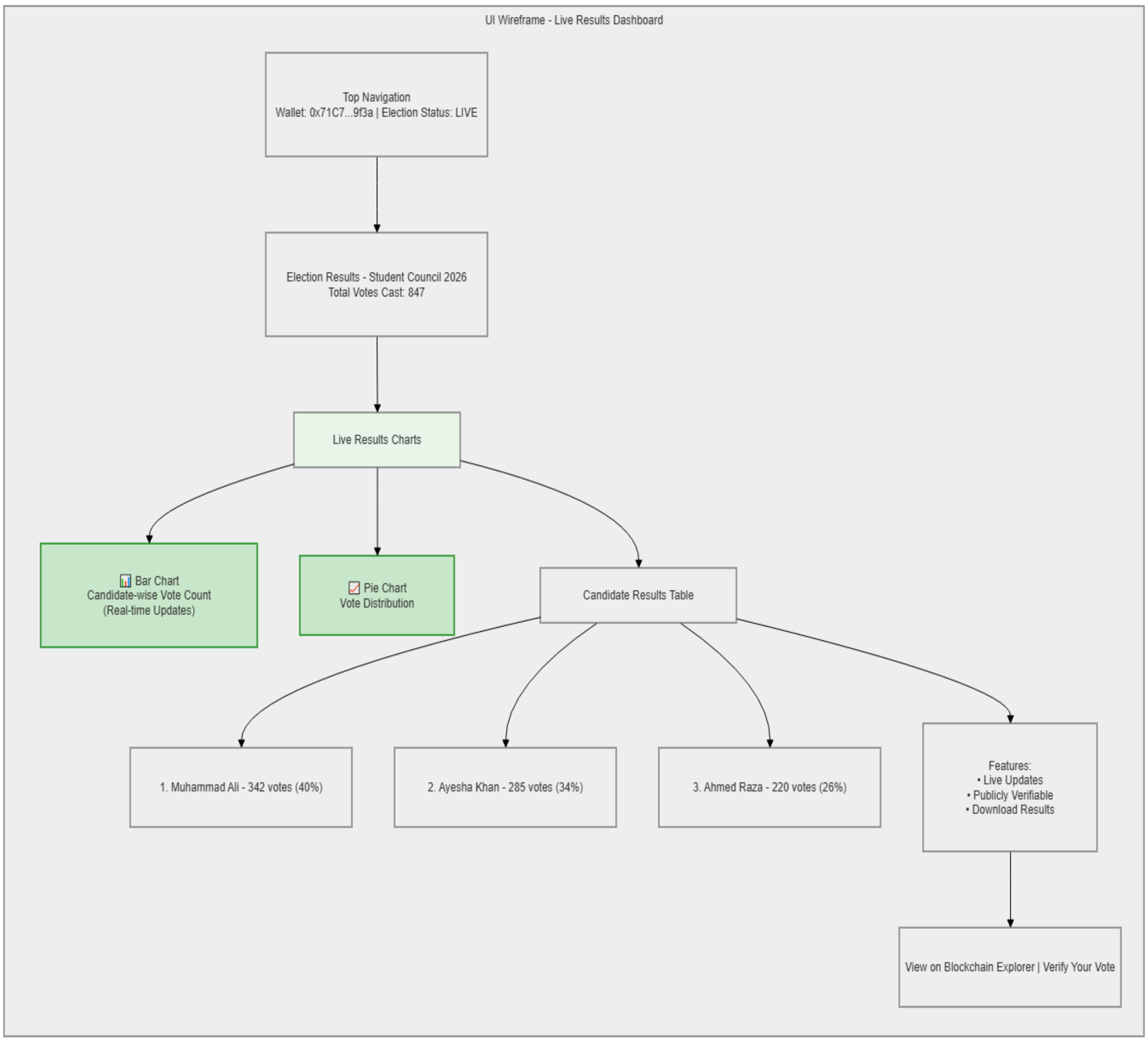


Figure 6.3: UI Wireframe – Live Results Dashboard

Description:

This wireframe represents the real-time results dashboard showing live vote counts, bar/pie charts, and a candidate ranking table. The design provides transparency with live updates from the blockchain



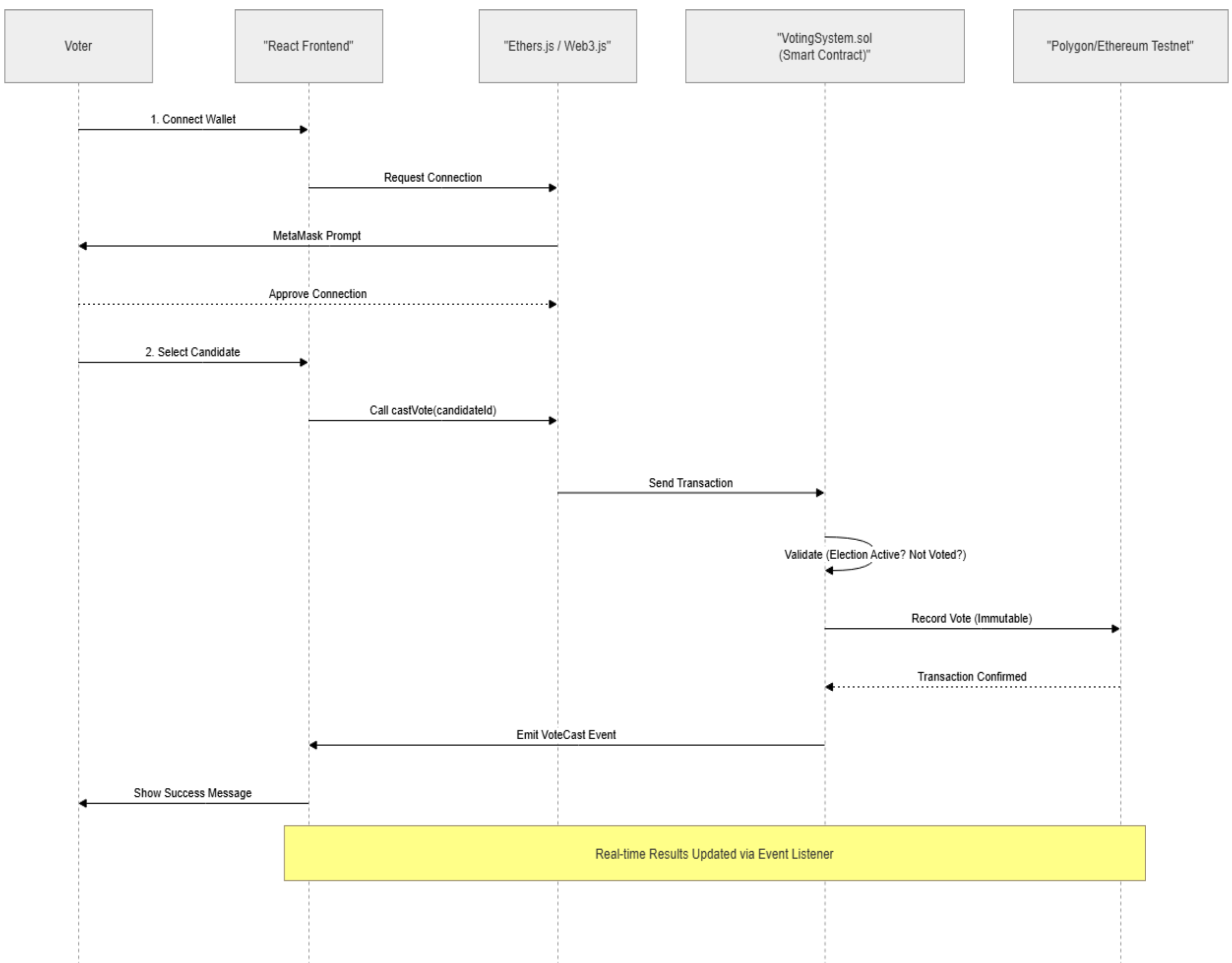
7. Interaction Design (Dynamic Modeling)

7.1 Sequence Diagram – Cast Vote Process

Description:

This sequence diagram illustrates the complete flow of the voting process from wallet connection to successful vote recording on the blockchain. It highlights the interaction between the user, frontend, Web3 library, smart contract, and the blockchain network.

Figure 7.1: Sequence Diagram for Cast Vote

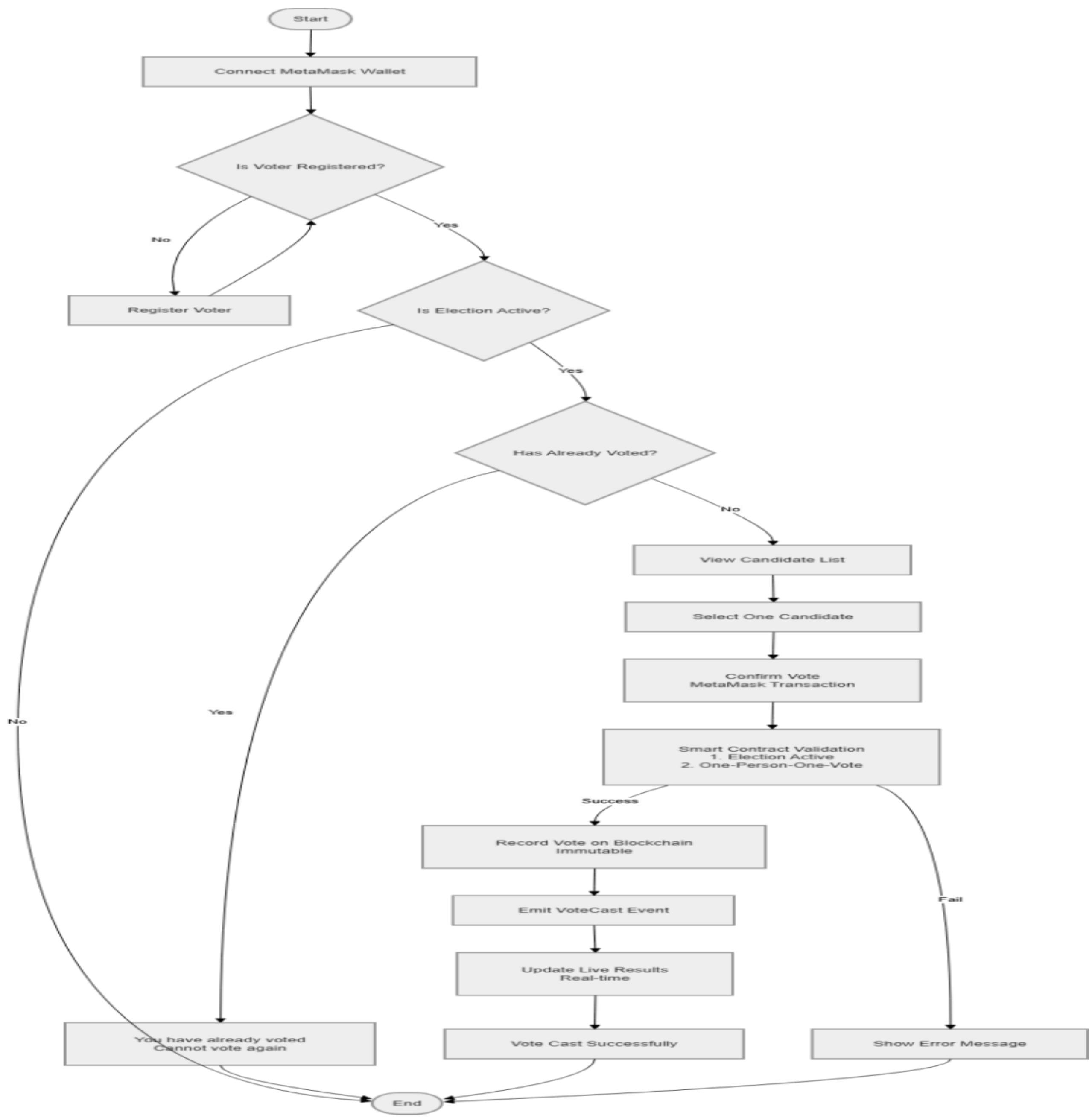


7.2 Activity Diagram – Overall Voting Flow (Optional)

Description:

This activity diagram shows the complete workflow of the voting process, including wallet connection, voter registration check, election status validation, one-person-one-vote enforcement, and successful vote recording on the blockchain.

Figure 7.2: Activity Diagram – Voting Process

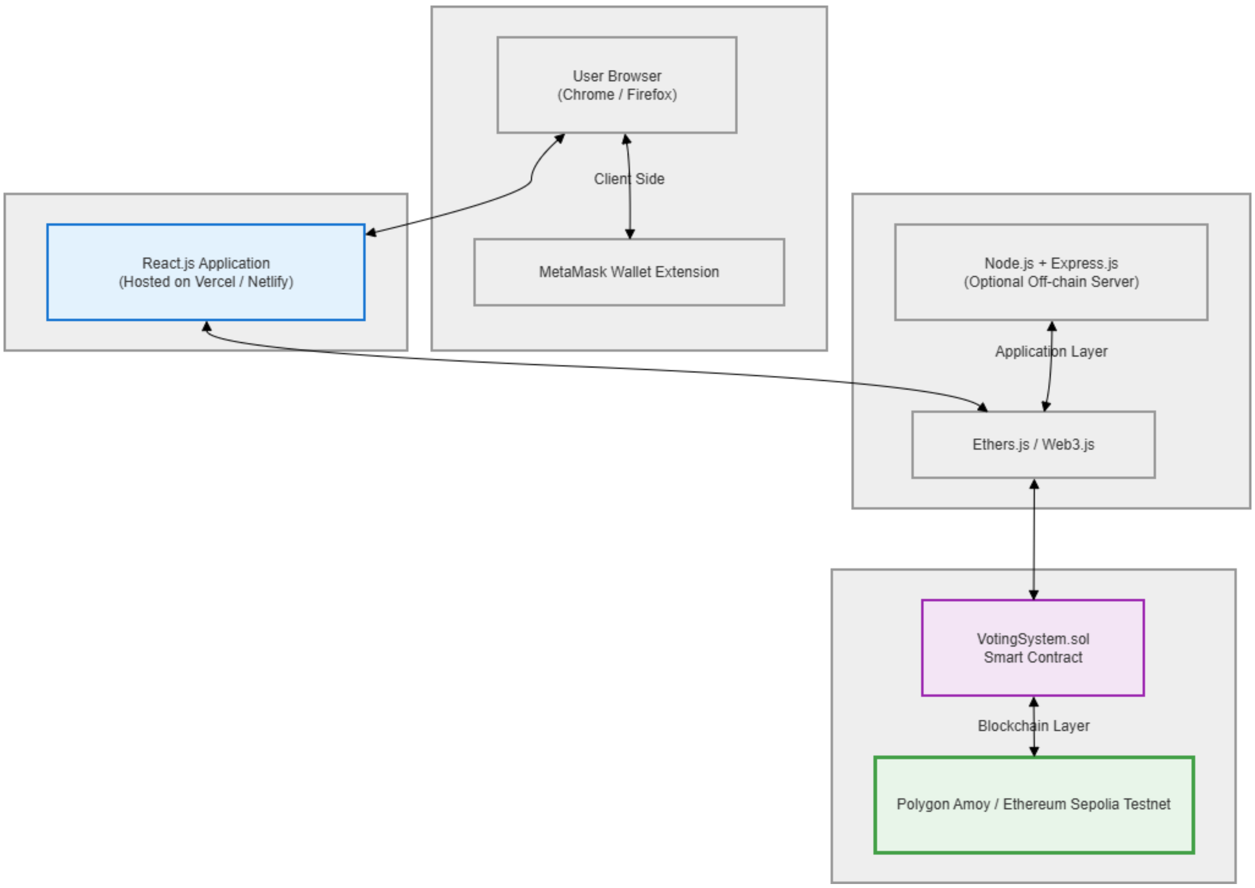


8. Deployment Architecture

Figure 8.1: Deployment Diagram

Description:

This deployment diagram shows how different components of the system are deployed. Users access the React.js frontend through their browser with MetaMask. The frontend interacts with the Solidity smart contract deployed on the Polygon/Ethereum testnet.



9. Testing Strategy

Smart Contract Unit Testing (Hardhat + Chai)

Integration Testing (Frontend + Blockchain)

Security & Gas Testing

10. Conclusion

This SDD provides a comprehensive and technically sound design for implementing a secure, transparent, and practical blockchain-based online voting system. All major diagrams and design decisions support the requirements defined in the SDD.